Python で作るフォルマント合成音声

erga0131

I.はじめに

皆さんは合成音声を聞いたことがあるでしょうか。ネット上ではAquesTalk（俗に言う「ゆっくりボイス」）や初音ミクが使われるようになって久しいですし、広義には郵便局の窓口で番号札の番号を読み上げるものだって合成音声と言えるでしょう。今や日常に浸透した合成音声ですが、今日はそれをPython というプログラミング言語を使って作ってしまおうという話です。なお、先に断っておくと、執筆時点では子音の発声については学習中なので今回は母音の発声のみ行います。また、今回の記事を書くきっかけが、かわいいかわいい美少女ロボットの足立レイ[i]にあるので、今回はちょっと機械っぽい美少女をイメージした声を作っていこうと思います。

頼まれてもいないのに私の好みを暴露したところで話を戻すと、合成音声と一口に言っても大きく分けて二種類のものがあります。[ii]一つは録音編集方式と言われ、単純に録音した音をつなげるものです。駅の放送などはこの方式です。元々が人間の声を録音しただけなので、単語自体は不自然さもなく聞こえるでしょう。一方で、録音した内容しか喋れないのと、完全に同じ人と環境を用意しない限り、同じ声で話せる内容を追加できないのが欠点です。例えば、私が通学時に通る目黒線の駅では、新横浜線開業に伴い追加された行き先の音声がそれまでのものと違っているようで、無理やりつなぎ合わせてちぐはぐに聞こえてしまっています[iii]。それに対して、登録していないワードも喋れるのがテキスト音声合成で、文字を入力してそれを音声にするものです。これも細分化できるのですが、ここでは二つだけ取り上げたいと思います。

一つ目は波形接続型音声合成です。これは有名なものだとVOCALOID が採用している方式です。最近流行りの Synthesizer V は統計的パラメトリック音声合成（SPSS）とのハイブリッドだったり、数年前かなり話題になった NEUTRINO は SPSS のみを使っていたり[iv]と手法的に若干下火ではありますが、合成音声キャラクターが流行るきっかけとなった初音ミクに使われていた方式ということで紹介しておきます。しかし、この手法では元となる声を出す人が必要です。今回、ロボット美少女の声を作りたいわけですが、残念ながら私にはロボット美少女の知り合いはいないのと、波形データを保存しておくとデータ量が多くなってしまうため、後述する理由に合わないので今回はやりません。

二つ目が今回やるフォルマント音声合成です。声も音なので、フーリエ変換をすればその波を各周波数の成分に分解できます。このときにできた周波数のピークの山を、小さい方から順に第一フォルマント（F1）、第二フォルマント（F2）…と呼びます。そして、人間は母音を F1,F2 を元に判別しているため、それの周波数を中心に合成して再生すれば母音の声に聞こえるはず、ということです。ちなみに、私は文系学部生なのでフーリエ級数展開とかストーン＝ワイエルシュトラスの定理とかは全くわかりませんが、Python で経済分析にも使う NumPy というライブラリを使えば原理を知らなくても数行のプログラムでフーリエ変換ができます。便利な世の中になりましたね。要するに、今回は音声認識技術の逆をやってみる、と考えれば間違っていないと思います。

II.実際にプログラムを作ってみる

前置きが長くなりましたが、実際にプログラムを作っていきましょう。特徴となる周波数とその倍音[v]に合わせればよいのですが、今回は「声（母音）は三角関数（sin）の足し算で簡単に作れます」

(https://www.nicovideo.jp/watch/sm13283644)という動画を参考にしつつ調整しまくって原型をとどめていない値を使います。

まず、音の波形を作る際に numpy が、その音を再生するのに sounddevice が必要なので、pip を使ってインストールします。

完了したら、最初にそれらのモジュールを読み込んでおきます。

```
import numpy as np
import sounddevice as sd
```

次に、早速ですが話す言葉を入れましょう。とは言っても、母音しか話せないので母音だけで話せる言葉にしておかないとエラーが出るはずです

```
text = ”あいうえお”
```

次に、変数

```
fs = 44100
d = 0.5
```

でサンプリング周波数と各母音の長さを定義します。サンプリング周波数は、アナログな波である音を PC 上で再現するために、一秒間に何回波を取得するかという値です。これが大きいほどよりアナログの波に近似できるので高音質になるはずです。今回は CD 音質を基準に 44.1kHz にしてみました。

そして、t を定義しておきます。これは、サンプリングする時刻を配列にしたものです。後で使います。

```
t = np.linspace(0, d, int(d * fs), endpoint=False)
```

そして、音を定義していきます。後の拡張性のため、基本周波数(F0、動画内ではベース音と言われている)を変数 freq に入れて、その整数倍の周波数と振幅を辞書型変数 vowel_params に格納することで各音を定義していきます。

```
freq=300
vowel_params = {
    "あ": {
        "freq": [freq*1, freq*2, freq*3, freq*4, freq*5],
        "amp":  [0.40, 0.15, 0.35, 0.30, 0.10]
    },
    "い": {
        "freq": [freq*1, freq*2, freq*10, freq*11],
        "amp":  [0.52, 0.03, 0.02, 0.02]
    },
    "う": {
        "freq": [freq*1, freq*2, freq*4, freq*5, freq*6],
        "amp":  [0.45, 0.20, 0.05, 0.05, 0.02]
    },
    "え": {
        "freq": [freq*1, freq*2, freq*3, freq*8, freq*9],
        "amp":  [0.20, 0.40, 0.03, 0.05, 0.01]
    },
    "お": {
        "freq": [freq*1, freq*2, freq*3, freq*4],
        "amp":  [0.35, 0.30, 0.25, 0.10]
    }
}
```

このように、周波数と振幅を定義したら、波形を生成する部分を作っていきます。

x_total が完成した波形をいれる空の配列です。

Python はインデント(字下げ)が重要なのでここからは注意してください。x= の行は freq))])までで一行です。

```
x_total = np.array([])
for char in text:
    freq = vowel_params[char]["freq"]
    x = sum([vowel_params[char]["amp"][i] * np.sin(2 *
np.pi * freq[i] * t) for i in range(len(freq))])
    x_total = np.concatenate((x_total, x))
```

ここからちょっとだけ波の話をすると、sin 波は振幅 A,距離 x,波長λ,周波数 f,時刻 t として

$$y(x,t) = Asin\left(\frac{2\pi}{\lambda}x - 2\pi ft\right)$$

と書けるため、スピーカーの振動板の位置を仮に x=0 と置けば、

$$y(0,t) = Asin(-2\pi ft) = -Asin(2\pi ft)$$

となります。負の符号は位相の反転を示しますが、音の聞こえとしては正の符号の場合と同じなので、マイナスを外します。そして、先程作った辞書にある母音の各周波数と振幅を一つずつ合成し(260Hz を振幅 0.07 で鳴らす sin 波を作る、520Hz を振幅 0.09 で鳴らす sin 波を作る、…)、それらを sum で一つの音に重ね合わせます。こうして一つの母音をが完成したら、最後に

```
sd.play(x_total, fs)
sd.wait()
```

concatenate で一つながりの波形につなぎ合わせて x_total として出力します。

最後に、上の二行を用いて、sounddevice で音を出力すれば完成です。

こうして、Python を使って母音っぽい音を生成できるようになりましたが、なぜパソコン上でプログラムをいじるだけのこの記事をわざわざ「無線工学研究会」の部誌に書いたかと言えば、これが無線通信(より詳しく言えば、携帯電話の通話)に関連している技術だからです。

かねてより、音声の波形をそのまま転送すると通信容量が大きくなってしまうため、携帯電話のように特に通信量にシビアなものを扱う際、データそのままよりも処理したデータ(今回の周波数と振幅の配列のようなもの)だけを送り、受け手で音声を合成すれば容量が少なくて済む、という考えがありました。実際に、今回作った音声をそのまま wav ファイル(波形をそのまま保存する形式)に出力

してみると 215 KB になりますが、プログラムは波形生成部分も含めても 2.19 KB と、容量が約 1/100 に収まっています。ちなみに、こういった経緯もあって、日本最初の合成音声は旧・郵政省電波研究所によって作られました。[vi]

III.おわりに

今回は母音の発声のみでしたが、子音も発生できるようにして五十音を言えるようにしたり、変数 freq で基本周波数を変えて声の高さを変えて歌えるようにしたりと、様々な可能性があると考えています。皆さんも、自分の PC で自分だけの「声」を作ってみてはいかがでしょうか？

---

[i] 詳しくは「足立レイ – メカニカルガール」 https://mechanicalgirl.jp/adachi-rei/ をご覧ください。足立レイの声もフォルマント合成で作られていて、その制作過程は「【足立レイ】中の人のいない合成音声作った【UTAU 音源配布】」(https://www.nicovideo.jp/watch/sm33386516)で公開されています

[ii] 「AI データ分析コラム【25】音声合成における AI 活用について」2025.03.04 更新. https://www.ntt-at.co.jp/column/ai-data-analysis/202503.html (2025/08/22 閲覧)による。

[iii] 「東急目黒線目黒駅 1 番線　接近放送(急行新横浜行)&メトロ 9000 系到着」 https://www.youtube.com/watch?v=QhRk68Ir2zw を見れば伝わると思います。
「まもなく　1 番線に(旧)」「東急新横浜線直通(新)」「急行(旧)」「新横浜行きが(新)」「6 両編成でまいります(旧)」ではないかと思いますが、新旧の声が混ざっています。

[iv] 「音声合成ソフトウェアの概要/初音ミク Wiki」(2025 年 06 月 02 日 15:22 更新) https://w.atwiki.jp/hmiku/pages/53500.html (2025/08/22 閲覧)による。

[v] F1-F2 図、だとかフォルマントとかでググれば図が載っています。この 2 つの周波数さえ再生すれば母音を判別できるかのように思えますが、それだけではただの sin 波にしか聞こえないので、それらの倍音も再現して声っぽく聞こえるようにします。

[vi] 「日本最初の合成音声」 https://www.youtube.com/watch?v=uXLkQMOk6as という動画で、実際の音声を聞くことができます。